

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – MULTIMODAL MINI-DEMO (TEXT-TO-IMAGE + SPEECH-TO-TEXT / TEXT-TO-SPEECH)

Course Code:	CSA6501	Course Title:	Generative AI and Large Language Models
CO Assessed:	CO4 – Precision (BL3, BL4,BL5)	Assessment:	Assessment Tool 3 - Multimodal Mini-Demo (Text-to-Image + Speech-to-Text / Text-to-Speech)
Weightage:	40% of CIA	Total Marks:	1 * 100= 100
CO4 Statement:	<i>Design and implement multimodal Generative AI applications integrating text, image, and speech models for engineering applications.(BL3, BL4,BL5)</i>		
Student Name:	A.Sai Rohit	Reg. No.:	192472144
Section / Batch:	CSE-AI	Date:	

Instructions to Students

- Answer 1 question1. Total: 100 marks.
- Implement the assigned multimodal pipeline.
- Use at least two modalities, including Text-to-Image and either Speech-to-Text or Text-to-Speech.
- Demonstrate the application with different inputs, not just one example.
- Show the generated image/audio/transcription clearly.
- Explain the model/API/tool used and its limitations.
- Demonstrate how the application handles different or imperfect inputs.
- Provide a short, polished live demonstration.
- Explain the relevance and quality of the generated outputs.

Code of Conduct

I A.Sai Rohit (192472144) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: A.Sai Rohit

Q18. Software Development Learning Tool

Build an application that generates a conceptual image explaining a software development topic such as *software testing* or *version control* and converts a student's spoken explanation into text.

1. INTRODUCTION

Software development concepts such as Software Testing, Version Control and Debugging can be difficult to understand using text alone. Visual representations can help students understand the workflow of a concept, while speech recognition allows students to explain their understanding verbally. The Software Development Learning Tool is a multimodal application developed using Python and Streamlit. The application uses two modalities: Text-to-Image and Speech-to-Text. The user enters a software development topic as text input, and the application generates a conceptual visual representation of the topic. The student can also upload a spoken explanation as an audio file, and the application converts the speech into text. Therefore, the application combines text, visual and speech modalities to provide an interactive learning experience.

2. OBJECTIVE

The main objective of this project is to develop a multimodal learning tool that helps students understand and explain software development concepts.

The objectives are:

- To accept software development topics as text input.
- To generate a conceptual image representing the selected topic.
- To convert a student's spoken explanation into text.
- To demonstrate the application using different inputs.
- To handle unclear or imperfect speech input.
- To provide an interactive and easy-to-use learning interface.

3. TECHNOLOGIES AND TOOLS USED

Programming Language:

Python

Web Application Framework:

Streamlit

Conceptual Image Generation:

Matplotlib

Speech-to-Text:

SpeechRecognition Python Library

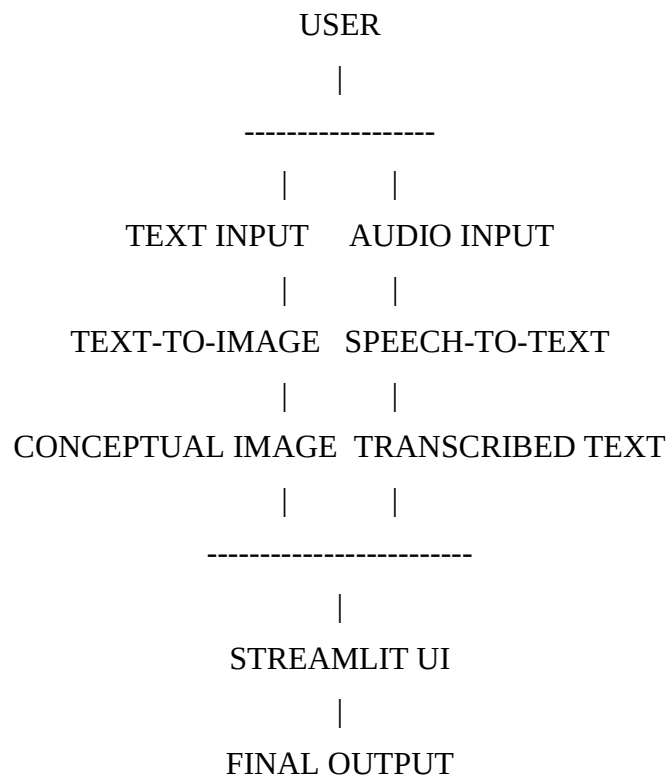
Speech Recognition Service:

Google Speech Recognition

Other Python Libraries:

Temporary file handling and OS utilities

4. SYSTEM ARCHITECTURE



5. WORKING PRINCIPLE

The application has two main processing pipelines.

Pipeline 1: Text-to-Image

The user enters a software development topic such as Software Testing, Version Control or Debugging. The application identifies the topic and selects the corresponding workflow. Matplotlib is then used to create a conceptual image using boxes and arrows.

Pipeline 2: Speech-to-Text

The student uploads an audio recording containing a spoken explanation. The application reads the audio using the SpeechRecognition library and sends it to Google Speech Recognition. The recognized speech is converted into text and displayed on the Streamlit interface.

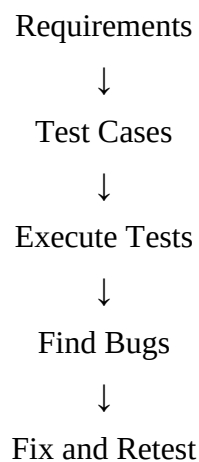
6. TEXT-TO-IMAGE IMPLEMENTATION

The Text-to-Image component accepts a software development topic from the user.

Example input:

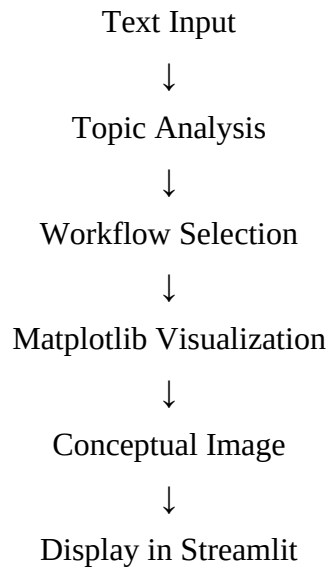
Software Testing

The application identifies the topic and generates the following conceptual workflow:



The workflow is represented visually using boxes and arrows.

Processing flow:



← → ↻ ⓘ localhost:8501 🔍 ☆ ⬇️ 🔄 🟢 School ⋮

Deploy ⋮



Software Development Learning Tool

A multimodal learning application that generates a conceptual image for software development topics and converts student speech into text.



1. Text-to-Image

Enter a software development topic:

Version Control

Generate Conceptual Image

Conceptual image generated successfully!

Conceptual image generated successfully!

Generated Conceptual Image

Conceptual Workflow: Version Control



```
graph LR; A[Write Code] --> B[Git Add]; B --> C[Git Commit]; C --> D[Git Push]; D --> E[Track Changes];
```

Topic: Version Control

1. Text-to-Image

Enter a software development topic:

Debugging

Generate Conceptual Image

Conceptual image generated successfully!

Generated Conceptual Image

Conceptual Workflow: Debugging



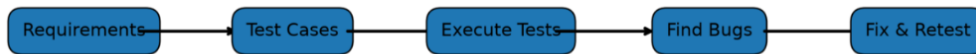
Debugging :

Generate Conceptual Image

Conceptual image generated successfully!

Generated Conceptual Image

Conceptual Workflow: Software Testing



7. SPEECH-TO-TEXT IMPLEMENTATION

The second modality of the application is Speech-to-Text.

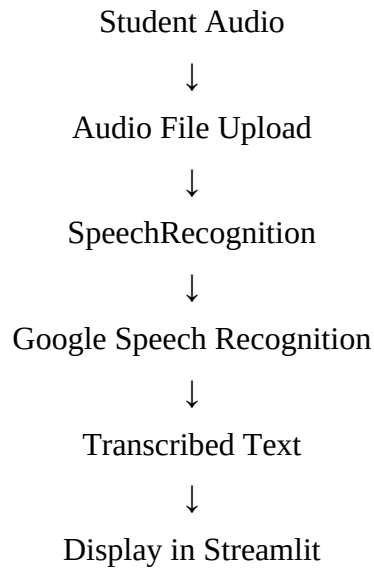
The student uploads an audio recording containing their explanation of a software development topic.

For example, the student can say:

"Software testing is the process of finding errors and ensuring that software works correctly."

The application processes the audio and converts the spoken explanation into text.

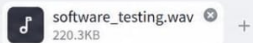
Processing flow:



2. Speech-to-Text

Upload an audio recording containing the student's explanation.

Upload audio file



▶ 0:00 / 0:05



Convert Speech to Text



Speech converted successfully!



Transcribed Explanation

Software testing is the process of finding errors and ensuring that software works correctly.

8. HANDLING IMPERFECT INPUT

The application was tested with different speech inputs to observe how the Speech-to-Text component handles imperfect input.

Examples of imperfect input include:

- Background noise.
- Unclear pronunciation.

- Very low audio volume.
- Long pauses.
- Incomplete speech.

When the speech is unclear and cannot be recognized, the application displays an appropriate error message:

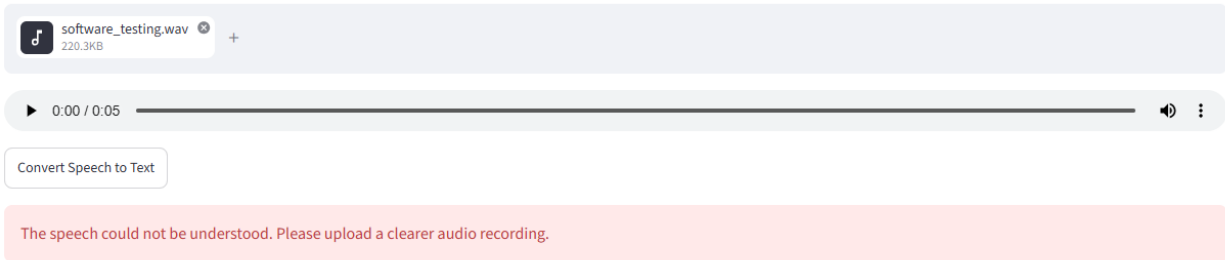
"The speech could not be understood. Please upload a clearer audio recording."

This prevents the application from failing silently and informs the user that a clearer recording is required.

2. Speech-to-Text

Upload an audio recording containing the student's explanation.

Upload audio file



The screenshot shows a web interface for speech-to-text. At the top, there's a header with a microphone icon and the title "2. Speech-to-Text". Below the header, a instruction says "Upload an audio recording containing the student's explanation." Underneath, there's a section labeled "Upload audio file". Inside this section, a file named "software_testing.wav" (220.3KB) is shown with a plus sign to its right. Below the file list is a progress bar showing "0:00 / 0:05" and a play button icon. Below the progress bar is a button labeled "Convert Speech to Text". At the bottom of the interface, a red error message box displays the text: "The speech could not be understood. Please upload a clearer audio recording."

9. HANDLING DIFFERENT TEXT INPUTS

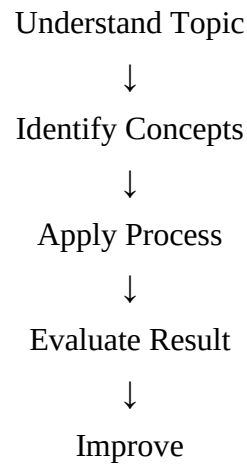
The application supports different software development topics.

For recognized topics such as:

- Software Testing
- Version Control
- Debugging

the application generates topic-specific workflows.

If an unknown topic is entered, the application does not crash. Instead, it generates a general conceptual workflow:



This demonstrates that the application can handle different inputs instead of depending on only one fixed example.

10. MODEL / API / TOOL USED

TEXT-TO-IMAGE

The application uses Python and Matplotlib to create conceptual visual representations. The entered topic is analyzed using keyword matching. Based on the topic, a predefined set of workflow steps is selected. Matplotlib then converts these steps into a visual diagram using text boxes and arrows. This approach provides a simple and clear educational representation of software development concepts.

SPEECH-TO-TEXT

The application uses the SpeechRecognition Python library along with Google Speech Recognition. The SpeechRecognition library captures and processes the uploaded WAV audio file. Google Speech Recognition is then used to recognize the spoken words and convert them into text. The resulting transcription is displayed directly in the Streamlit application.

11. RELEVANCE AND QUALITY OF OUTPUTS

The generated outputs are relevant to the selected software development topics. For Software Testing, the visual output represents the testing process from requirements and test cases to identifying bugs and retesting. For Version Control, the visual output represents the process of writing code, adding changes, committing changes, pushing changes and tracking modifications. For Debugging, the visual output represents the process of running a program, finding an error, analyzing the cause, fixing the code and testing again. The Speech-to-Text output provides a textual representation of the student's spoken explanation. This can help students check and review their technical explanations. The outputs are simple, clear and suitable for an educational demonstration.

12. LIMITATIONS

The application has the following limitations:

- The conceptual images are based on predefined workflows.
- The visual output is an educational diagram rather than a fully generative AI image.
- Only selected software development topics have specific workflows.
- Unknown topics use a general workflow.
- Speech recognition accuracy can decrease because of background noise.
- Unclear pronunciation can affect transcription accuracy.
- Internet connectivity is required for Google Speech Recognition.
- The current implementation accepts WAV audio files for reliable processing.

13. RESULTS AND DISCUSSION

The Software Development Learning Tool successfully demonstrated a multimodal pipeline using Text-to-Image and Speech-to-Text. The Text-to-Image component generated different conceptual visual outputs for Software Testing, Version Control and Debugging. The Speech-to-Text component converted uploaded spoken explanations into readable text. The application was tested using different inputs rather than only one example. This demonstrated that the system can generate different visual outputs and process different spoken explanations. The application was also tested with imperfect speech. When the speech could not be understood, the application displayed an appropriate error message and requested a

clearer recording. The results show that combining visual representation and speech recognition can make software development learning more interactive and accessible.

14. LIVE DEMONSTRATION PROCEDURE

Step 1:

Open the Software Development Learning Tool in the browser.

Step 2:

Enter:

Software Testing

Click "Generate Conceptual Image".

Show the generated conceptual workflow.

Step 3:

Upload the WAV audio containing the explanation of Software Testing.

Click "Convert Speech to Text".

Show the generated transcription.

Step 4:

Change the topic to:

Version Control

Generate the conceptual image and show the different workflow.

Step 5:

Upload the Version Control audio and convert it into text.

Step 6:

Change the topic to:

Debugging

Generate the conceptual image and show the output.

Step 7:

Upload the Debugging audio and display the transcription.

Step 8:

Finally, demonstrate an unclear or imperfect audio input and show how the application displays an error message.

15. ADVANTAGES

- Combines two different modalities in one application.
- Provides visual representations of software development concepts.
- Allows students to explain concepts using speech.
- Converts spoken explanations into readable text.
- Simple and easy-to-use Streamlit interface.
- Supports multiple software development topics.
- Provides feedback when speech cannot be recognized.

16. CONCLUSION

The Software Development Learning Tool was successfully developed as a multimodal application using Text-to-Image and Speech-to-Text. The application accepts a software development topic as text input and generates a conceptual visual representation. It also accepts a student's spoken explanation and converts it into text. The system was demonstrated using multiple topics including Software Testing, Version Control and Debugging. It was also tested with imperfect speech to demonstrate error handling. The project demonstrates how multimodal technologies can be combined to create an interactive educational tool that supports both visual learning and spoken explanations.

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Pipeline Implementation	30%	Correctly implements a working text-to-image AND speech-to-text/text-to-speech pipeline	Both implemented with minor issues	Only one modality implemented correctly	Neither modality functions correctly
Output Quality & Relevance	25%	Generated outputs are high-quality and clearly relevant to inputs	Outputs are mostly relevant	Outputs are of inconsistent quality	Outputs are largely irrelevant or poor quality
Input Robustness	20%	Handles varied inputs robustly (different prompts/audio clips)	Handles common inputs well	Handles only the simplest inputs	Fails on basic inputs
Demo Polish	15%	Polished, engaging demo of both modalities	Clear demo with minor polish issues	Basic or rough demo	No usable demo presented
Tool/Model Explanation	10%	Clear explanation of models/tools used and their limitations	Adequate explanation	Minimal explanation	No explanation provided

Marks Evaluation:

Question No.	Pipeline Implementationy (30%)	Output Quality & Relevance (25%)	Input Robustness (20%)	Demo Polish (15%)	Tool/Model Explanation (10%)	Total Marks (100)
Q1						
Overall Total (100)						